

Лабораторная работа 7

Введение в работу с данными

Ланцова Я. И.

Российский университет дружбы народов, Москва, Россия

Информация

- Ланцова Яна Игоревна
- студентка
- Российский университет дружбы народов

Основной целью работы является освоение специализированных пакетов Julia для обработки данных.

1. Используя JupyterLab, повторите примеры. При этом дополните графики обозначениями осей координат, легендой с названиями траекторий, названиями графиков и т.п.
2. Выполните задания для самостоятельной работы.

Выполнение лабораторной работы

Выполнение лабораторной работы

```
[2]: # Считывание данных и их запись в структуру!
# = CSV.File("~/Users/yana/Downloads/programminglanguages.csv") |> DataFrame

[2]: 73x2 DataFrame                                     48 rows omitted
  Row  year  language
    int64  String31
1    1951 Regional Assembly Language
2    1952 Autocode
3    1954 IPL
4    1955 FLOW-MATIC
5    1957 FORTRAN
6    1957 COMTRAN
7    1958 LISP
8    1958 ALGOL 58
9    1959 FACT
10   1959 COBOL
11   1959 RPG
12   1962 APL
13   1962 Simula
:      :
:      :
```

Рис. 1: Считывание данных

Выполнение лабораторной работы

```
[4]: # Функция определения по названию языка программирования года его создания:
function language_created_year(P, language::String)
    loc = findfirst(P[1,2].==language)
    return P[loc,1]
end

[4]: language_created_year (generic function with 1 method)

[5]: # Пример вызова функции и определение даты создания языка Python:
language_created_year(P,"Python")
# Пример вызова функции и определение даты создания языка Julia:
language_created_year(P,"Julia")

[5]: 2012

[9]: # Функция определения по названию языка программирования
# года его создания (без учёта регистра):
function language_created_year_v2(P, language::String)
    loc = findfirst(lowercase(P[:,2]).==lowercase(language))
    return P[loc,1]
end

# Пример вызова функции и определение даты создания языка Julia:
language_created_year_v2(P,"Julia")

[9]: 2012

[10]: # Построение считывание данных с указанием разделителя:
Tx = readdelim("/Users/yana/Downloads/programminglanguages.csv", ",")

[10]: 74x2 Matrix{Any}:
"year"  "Language"
1951    "Regional Assembly Language"
1952    "Autocode"
1954    "IPL"
1955    "PLD-MATIC"
```

Рис. 2: Считывание данных

Выполнение лабораторной работы

```
[11]: # Пример записи данных в текстовый файл с разделителем ","
writedle("programming_languages_data.txt", Tx, ',')
# Пример записи данных в текстовый файл с разделителем "\n":
writedle("programming_languages_data2.txt", Tx, "\n")

[12]: # Построчное считывание данных с указанием разделителя:
P_new_delim = readle("programming_languages_data2.txt", "-")
```

```
[12]: 74x2 Matrix(Any):
      "year"  "language"
1951         "Assembly Language"
1952         "Autocode"
1954         "IPL"
1955         "FLOW-MATIC"
1957         "FORTRAN"
1957         "COMTRAN"
1958         "LISP"
1958         "ALGOL 58"
1959         "FACT"
1959         "COBOL"
1959         "BASIC"
1962         "APL"

2003         "Scala"
2005         "R"
2006         "PowerShell"
2007         "Clojure"
2009         "Go"
2010         "Rust"
2011         "Dart"
2011         "Kotlin"
2011         "Perl"
```

Рис. 3: Запись данных в файл

Выполнение лабораторной работы

Словари

```
[13]: # инициализация словаря:
dict = Dict{Integer, Vector{String}}()

[13]: Dict{Integer, Vector{String}}()

[14]: # инициализация словаря:
dict2 = Dict{String, Vector{String}}()

[14]: Dict{String, Vector{String}}()

[16]: # Заполнение словаря данными:
for i = 1:size(P,1)
    year, lang = P[i,:1]
    if year in keys(dict)
        dict[year] = push!(dict[year], lang)
    else
        dict[year] = [lang]
    end
end

[17]: dict[2003]

[17]: 4-element Vector{String}:
"Groovy"
"Scala"
"Groovy"
"Scala"
```

Рис. 4: Словари

Выполнение лабораторной работы

DataFrames

```
[18]: # Подгружаем пакет DataFrames:
using DataFrames
# Задаём переменную со структурой DataFrame:
df = DataFrame{year = Pl{,1}, language = Pl{,2}}

[18]: 73x2 DataFrame                                     48 rows omitted
```

Row	year	language
	Int64	String{31}
1	1951	Regional Assembly Language
2	1952	Autocode
3	1954	IPL
4	1955	FLOW-MATIC
5	1957	FORTRAN
6	1957	COMTRAN
7	1958	LISP
8	1958	ALGOL 58
9	1959	FACT
10	1959	COBOL
11	1959	RPG
12	1962	APL
13	1962	Simula
⋮	⋮	⋮
62	2003	Scala
63	2005	F#

Рис. 5: DataFrames

Выполнение лабораторной работы

```
[20]: # Получение статистических сведений о df:
describe(df)

[20]: 2x7 DataFrame
      Row variable  mean   min   median  max  nmissing  eltype
      Symbol Union... Any   Union... Any   int64  DataType
1  year  1982.99  1961    1986.0  2014      0  int64
2  language      ALGOL 58      dbase III      0  String31
```

Рис. 6: DataFrames

Выполнение лабораторной работы

RDatasets

```
[21]: # Загружаем пакет RDatasets:
      using RDatasets
      # Задаем структуру данных в виде набора данных:
      iris = dataset("datasets", "iris")
```

[21]: 150x5 DataFrame 125 rows omitted

Row	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
	Float64	Float64	Float64	Float64	Cat...
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
7	4.8	3.4	1.4	0.3	setosa
8	5.0	3.4	1.5	0.2	setosa
9	4.4	2.9	1.4	0.2	setosa
10	4.9	3.1	1.5	0.1	setosa
11	5.4	3.7	1.5	0.2	setosa
12	4.8	3.4	1.6	0.2	setosa
13	4.8	3.0	1.4	0.1	setosa
:	:	:	:	:	:

Рис. 7: RDatasets

Выполнение лабораторной работы

```
[22]: # Определение типа переменной:
      typeof(iris)

[22]: DataFrame

[23]: describe(iris)

[23]: 5x7 DataFrame
```

Row	variable	mean	min	median	max	nmissing	eltype
	Symbol	Union...	Any	Union...	Any	Int64	DataType
1	SepaLength	5.84333	4.3	5.8	7.9	0	Float64
2	SepaWidth	3.05733	2.0	3.0	4.4	0	Float64
3	PetaLength	3.758	1.0	4.35	6.9	0	Float64
4	PetaWidth	1.19933	0.1	1.3	2.5	0	Float64
5	Species		setosa		virginica	0	CategoricalValue{String, UInt8}

Рис. 8: RDatasets

Выполнение лабораторной работы

Работа с переменными отсутствующего типа (Missing Values)

```
[24]: # Отсутствующий тип:  
a = missing  
typeof(a)  
[24]: Missing  
  
[25]: # Определение перечня продуктов:  
foods = ["apple", "cucumber", "tomato", "banana"]  
# Определение калорий:  
calories = [missing, 47, 22, 185]  
[25]: 4-element Vector{Union{Missing, Int64}}:  
missing  
47  
22  
185  
  
[26]: # Определение типа переменной:  
typeof(calories)  
[26]: Vector{Union{Missing, Int64}} (alias for Array{Union{Missing, Int64}, 1})  
  
[28]: # Подключение пакета Statistics:  
using Statistics  
# Определение среднего значения:  
mean(calories)  
# Определение среднего значения без значений с отсутствующим типом:  
mean(skipmissing(calories))  
[28]: 58.0
```

Рис. 9: Работа с переменными отсутствующего типа (Missing Values)

Выполнение лабораторной работы

Кластеризация данных. Метод k-средних

```
[36]: # Загрузка данных:  
houses = CSV.File("/Users/yana/Downloads/houses.csv") |> DataFrame
```

[36]: 985x12 DataFrame

960 rows omitted

Row	street String	city String15	zip Int64	state String3	beds Int64	baths Int64	sq_ft Int64	type String15	sale_date String31	price Int64	latitude Float64	longitude Float64
1	3526 HIGH ST	SACRAMENTO	95838	CA	2	1	836	Residential	Wed May 21 00:00:00 EDT 2008	59222	38.6319	-121.435
2	51 OMAHA CT	SACRAMENTO	95823	CA	3	1	1167	Residential	Wed May 21 00:00:00 EDT 2008	68212	38.4789	-121.431
3	2796 BRANCH ST	SACRAMENTO	95815	CA	2	1	796	Residential	Wed May 21 00:00:00 EDT 2008	68880	38.6183	-121.444
4	2805 JANETTE WAY	SACRAMENTO	95815	CA	2	1	852	Residential	Wed May 21 00:00:00 EDT 2008	69307	38.6168	-121.439
5	6001 MCMAHON DR	SACRAMENTO	95824	CA	2	1	797	Residential	Wed May 21 00:00:00 EDT 2008	81900	38.5195	-121.436
6	5828 PEPPERMILL CT	SACRAMENTO	95841	CA	3	1	1122	Condo	Wed May 21 00:00:00 EDT 2008	89921	38.6626	-121.328
7	6048 OGDEN NASH WAY	SACRAMENTO	95842	CA	3	2	1104	Residential	Wed May 21 00:00:00 EDT 2008	90895	38.6817	-121.352
8	2561 19TH AVE	SACRAMENTO	95820	CA	3	1	1177	Residential	Wed May 21 00:00:00 EDT 2008	91002	38.5351	-121.481
9	11150 TRINITY RIVER DR Unit 114	RANCHO CORDOVA	95670	CA	2	2	941	Condo	Wed May 21 00:00:00 EDT 2008	94905	38.8212	-121.271

Рис. 10: Кластеризация данных. Метод k-средних

Выполнение лабораторной работы

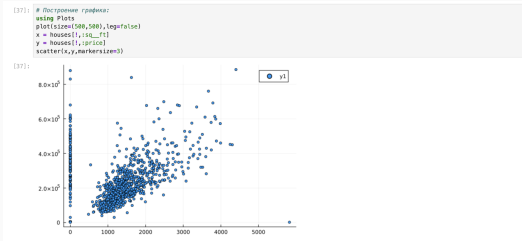


Рис. 11: Кластеризация данных. Метод k-средних

Выполнение лабораторной работы

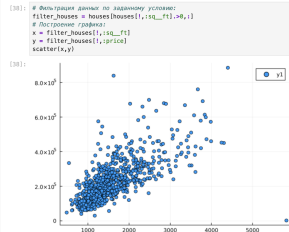


Рис. 12: Кластеризация данных. Метод k-средних

Выполнение лабораторной работы

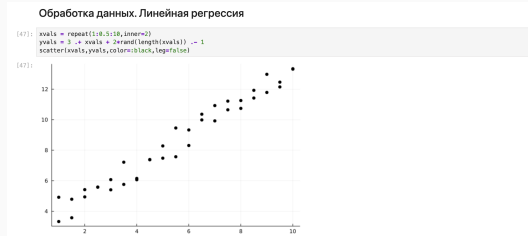


Рис. 13: Обработка данных. Линейная регрессия

Выполнение лабораторной работы

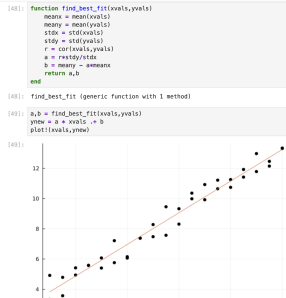


Рис. 14: Обработка данных. Линейная регрессия

Задание 1

Выполнение лабораторной работы

Задания для самостоятельного выполнения

Задание 1

Используйте Clustering.J для кластеризации на основе k-средних. Сделайте точечную диаграмму полученных кластеров. Подсказка: вам нужно будет проиндексировать фрейм данных, преобразовать его в массив и транспонировать.

```
[52]: using Datasets
iris = dataset("datasets", "iris")
```

```
[52]: 150x5 DataFrame
```

125 rows omitted

Row	SepalLength	SepalWidth	Petal.Length	Petal.Width	Species
	Float64	Float64	Float64	Float64	Cat...
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
7	4.6	3.4	1.4	0.3	setosa
8	5.0	3.4	1.5	0.2	setosa
9	4.4	2.9	1.4	0.2	setosa
10	4.9	3.1	1.5	0.1	setosa
11	5.4	3.7	1.6	0.2	setosa
12	4.8	3.4	1.6	0.2	setosa

Рис. 15: Кластеризация

Выполнение лабораторной работы

```
[55]: # Загрузка данных Iris
iris = dataset("datasets", "Iris")

# Просмотр структуры данных
print(f"Размеры данных Iris: ", size(iris))
print(f"Первые 5 строк:")
print(first(iris, 5))
print(f"Имена столбцов:")
print(names(iris))
print(f"Уникальные виды ирисов:")
print(unique(iris.Species))

Размеры данных Iris: (150, 5)

Первые 5 строк:
#5 DataFrame
  Row  SepalLength  SepalWidth  PetalLength  PetalWidth  Species
   Float64      Float64      Float64      Float64      Cat...
1      5.1         3.5         1.4         0.2  setosa
2      4.9         3.0         1.4         0.2  setosa
3      4.7         3.2         1.3         0.2  setosa
4      4.6         3.1         1.5         0.2  setosa
5      5.0         3.6         1.4         0.2  setosa

Имена столбцов:
["SepalLength", "SepalWidth", "PetalLength", "PetalWidth", "Species"]

Уникальные виды ирисов:
["setosa", "versicolor", "virginica"]
```

Рис. 16: Загрузка и просмотр данных

Выполнение лабораторной работы

```
[56]: # Подготовка данных для кластеризации
      # Берем только числовые признаки (первые 4 столбца)
      iris_features = iris.data

      # Преобразуем DataFrame в матрицу и транспонируем
      # (Clustering.jl ожидает данные в виде матрицы признаков × наблюдений)
      iris_matrix = Matrix{Float64}(iris_features)
      println("Размерность матрицы признаков: ", size(iris_matrix))

      Размерность матрицы признаков: (4, 150)

[57]: # Выполним кластеризацию методом K-средних
      k = 3 # Мы знаем, что в данных 3 вида ирисов
      result = kmeans(iris_matrix, k)

      # Получаем метки кластеров
      cluster_labels = assignments(result)

      # Получаем центры кластеров
      centroids = result.centers

      println("Кластеризация завершена!")
      println("Количество кластеров: ", k)
      println("Размеры каждого кластера: ", counts(result))
      println("Итераций выполнено: ", result.iterations)
      println("Общая стоимость (сумма квадратов расстояний): ", round(result.totalcost, digits=3))

      Кластеризация завершена:
      Количество кластеров: 3
      Размеры каждого кластера: [50, 39, 61]
      Итераций выполнено: 7
      Общая стоимость (сумма квадратов расстояний): 78.856
```

Рис. 17: Подготовка данных

Выполнение лабораторной работы

```
[58]: # Создание точечной диаграммы (scatter plot)
# Используем два наиболее информативных признака для визуализации
# SepalLength и SepalWidth (длина и ширина чашелистика)

# Создаем цветовую палитру для кластеров
colors = ['red', 'blue', 'green', 'orange', 'purple'] # больше цветов на всякий случай
cluster_colors = [colors[label] for label in cluster_labels]

# Создаем точечную диаграмму
scatter(Iris.SepalLength, Iris.SepalWidth,
        color=cluster_colors,
        markerstrokewidth=0.5,
        markerize=6,
        alpha=0.7,
        legend=False,
        xlabel="Sepal Length (cm)",
        ylabel="Sepal Width (cm)",
        title="Кластеризация Iris методом k-средних (k=3)",
        grid=True,
        size=(800, 600))
```

Рис. 18: Создание диаграммы

Выполнение лабораторной работы



Рис. 19: Диаграмма

Выполнение лабораторной работы

```
% Добавляем центры кластеров
for i in 1:k
    scatter([centroids[i, 1]], [centroids[i, 2]],
            color=colors[i],
            markersize=12,
            marker='star',
            markerstrokewidth=2,
            label='%центр кластера %i')
end

% Добавляем легенду для центров
plot([legend:topright])
```

Рис. 20: Создание диаграммы

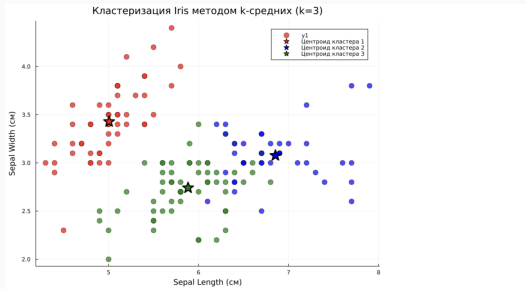


Рис. 21: Диаграмма

Выполнение лабораторной работы

```
[61]: # Создаем DataFrame с результатами кластеризации
using DataFrames
iris_with_clusters = copy(iris)
iris_with_clusters[:, :Cluster] = cluster_labels

# Функция для создания матрицы парных диаграмм
function pairplot_matrix(df)
    features = ["SepalLength", "SepalWidth", "PetalLength", "PetalWidth"]
    n = length(features)

    plot_array = []

    for i in 1:n
        for j in 1:n
            if i == j
                # Диагональ - гистограмма распределения
                p = histogram(df[:, features[i]],
                              color=:gray, alpha=0.7,
                              legend=false,
                              title=features[i],
                              grid=false)
            else
                # Вспомогательные элементы - scatter plots
                p = scatter(df[:, features[j]], df[:, features[i]],
                             color=:cluster_colors,
                             markerstrokewidth=0,
                             markersize=4,
                             alpha=0.6,
                             legend=false,
                             grid=false)
            end
            push!(plot_array, p)
        end
    end

    return plot(plot_array..., layout=(n, n), size=(1000, 1000))
end
```

Рис. 22: Создание итогового DataFrame

Выполнение лабораторной работы

```
# Создаем и сохраняем матрицу парных диаграмм
pairplot = pairplot_matrix(iris_with_clusters)
save_fig(pairplot, "iris_pairplot_matrix.png")
print("\nМатрица парных диаграмм сохранена как 'iris_pairplot_matrix.png'")

# Сравнение кластеров с реальными видами ирисов
print("\nСравнение кластеров с реальными видами:")

# Создаем таблицу сопоставления
real_species = iris.Species
cluster_df = DataFrame(RealSpecies=real_species, Cluster=cluster_labels)

Матрица парных диаграмм сохранена как 'iris_pairplot_matrix.png'
```

Сравнение кластеров с реальными видами:

```
[61]: 150x2 DataFrame
```

125 rows omitted

Row	RealSpecies	Cluster
	Cat...	Int64
1	setosa	1
2	setosa	1
3	setosa	1
4	setosa	1
5	setosa	1
6	setosa	1
7	setosa	1
8	setosa	1
9	setosa	1
10	setosa	1
11	setosa	1

Рис. 23: Создание итогового DataFrame

Выполнение лабораторной работы

```
1: # Расчет инерции (суммы квадратов расстояний до центров)
   println("Информация о кластерах:")
   for i in 1:k
       cluster_points = iris_matrix[i, cluster_labels.== i]
       cluster_size = size(cluster_points, 2)

       if cluster_size > 0
           # Среднее расстояние до центра
           distances = sqrt(sum((cluster_points[i, j] - centroids[i, 1]).^2))
               for j in 1:cluster_size
           avg_distance = round(mean(distances), digits=3)
           std_distance = round(std(distances), digits=3)

           println("Кластер $i: $cluster_size образцов, среднее расстояние до центра: $avg_distance ± $std_distance")
       end
   end
```

Информация о кластерах:
Кластер 1: 30 образцов, среднее расстояние до центра: 0.462 ± 0.269
Кластер 2: 39 образцов, среднее расстояние до центра: 0.732 ± 0.345
Кластер 3: 61 образцов, среднее расстояние до центра: 0.731 ± 0.388

Рис. 24: Вывод информации о кластерах

Задание 2

Выполнение лабораторной работы

задание 2

часть 1

Пусть регрессионная зависимость является линейной. Матрица наблюдений факторов X имеет размерность $N \times 3$ `randn(N, 3)`, массив результатов $N \times 1$, регрессионная зависимость является линейной. Найдите МНК-оценку для линейной модели.

– Сравните свои результаты с результатами использования `lsq` из `MultivariateStats.jl` (посмотрите документацию).

– Сравните свои результаты с результатами использования регулярной регрессии наименьших квадратов из `GLM.jl`. Подсказка. Создайте матрицу данных X_2 , которая добавляет столбец единиц в начало матрицы данных, и решите систему линейных уравнений.

Объясните с помощью теоретических выкладок.

```
3|: # Генерация данных
N = 1000
X = randn(N, 3)
a0 = rand{3}
y = X * a0 + 0.1 * randn(N)

# МНК-оценка вручную (с добавлением столбца единиц для свободного члена)
X2 = hcat{ones(N), X} # Добавляем столбец единиц – это важно для интерцепта!
a_mnk = (X2' * X2) \ (X2' * y) # Решение системы (X'X)a = X'y
println("МНК-оценка (ручная): ", a_mnk)

# Сравнение с MultivariateStats.jl (функция `lsq`)
a_llsq = lsq(X, y, bias=true) # bias=true добавляет константу (столбец единиц)
println("МНК-оценка (MultivariateStats.jl, lsq): ", a_llsq)

МНК-оценка (ручная): [6.0337110182196254e-5, 0.4716359063598927, 0.48730003271456, 0.3845485718561637]
МНК-оценка (MultivariateStats.jl, lsq): [0.47163590635989283, 0.48730003271455996, 0.38454857185616375, 6.033711018219967e-5]
```

Рис. 25: Генерация данных и ручная оценка

Выполнение лабораторной работы

```
# Сравнение с GLM.jl (регулярная линейная регрессия)
# Создаем DataFrame для GLM (если не установлен – установите пакет DataFrames)
using DataFrames
df = DataFrame(X1=x[1,1], X2=x[1,2], X3=x[1,3], y=y)
model_glm = lm(@formula(y ~ X1 + X2 + X3), df)
a_glm = coef(model_glm)
println("МНК-оценка (GLM.jl): ", a_glm)

# Проверка совпадения результатов
println("Сравнение результатов:")
println("Разница между ручным МНК и llsq: ", norm(a_mnk - a_llsq))
println("Разница между ручным МНК и GLM: ", norm(a_mnk - a_glm))

# Теоретическое объяснение
println("====")
pstr = "Матрица X2 = [1 X] позволяет оценить модель y = β₀ + β₁x₁ + β₂x₂ + β₃x₃."
pstr = "Теоретически, МНК-оценка находится как a = (X2'X2)⁻¹X2'y."
pstr = "Это стандартный метод решения overdetermined системы уравнений."
println(pstr)

МНК-оценка (GLM.jl): [6.833711818220072e-5, 0.47163500635089283, 0.48730083271455906, 0.38454857105616375]

Сравнение результатов:
Разница между ручным МНК и llsq: 0.6172665123619459
Разница между ручным МНК и GLM: 1.368473830665185e-16
Матрица X2 = [1 X] позволяет оценить модель y = β₀ + β₁x₁ + β₂x₂ + β₃x₃.
Теоретически, МНК-оценка находится как a = (X2'X2)⁻¹X2'y.
Это стандартный метод решения overdetermined системы уравнений.
```

Рис. 26: Сопоставление результатов

Выполнение лабораторной работы

часть 2

Найдите линию регрессии, используя данные (X, y) . Постройте график (X, y) , используя точечный график. Добавьте линию регрессии, используя `abline()`. Добавьте заголовок «График регрессии» и подпишите оси x и y .

```
[1] # Генерация данных
X = rand(100)
y = 2 * X + 0.1 * randn(100)

# Регрессия (одна переменная, с константой)
X2 = hcat(ones(100), X)
a_mnk_id = (X2' * X2) \ (X2' * y)
println("Оценки коэффициентов (одномерная регрессия): ", a_mnk_id)

# Построение графика
p = scatter(X, y, label="Данные", title="График регрессии", xlabel="x", ylabel="y")

# Рассчитываем значения линии регрессии
x_line = range(minimum(X), maximum(X), length=100)
y_line = a_mnk_id[1] .+ a_mnk_id[2] .* x_line

# Добавляем линию регрессии
plot!(p, x_line, y_line, label="Линия регрессии", color=:red, linewidth=2)

# Отображаем график
display(p)

Оценки коэффициентов (одномерная регрессия): [0.012281122293427695, 1.9847617324640696]
```

Рис. 27: Построение линии регрессии

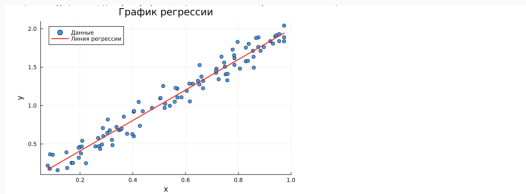


Рис. 28: График регрессии

Задание 3

Выполнение лабораторной работы

задание 3

Описание модели ценообразования биномиальных опционов можно найти на стр. https://en.wikipedia.org/wiki/Binomial_options_pricing_model.

Постройте траекторию возможных цен на акции:

- S — начальная цена акции;
- T — длина биномиального дерева в годах;
- n — количество периодов;
- $\lambda = T/n$ — длина одного периода;
- σ — волатильность акции;
- r — годовая процентная ставка;
- $u = \exp(r\lambda + \sigma\sqrt{\lambda})$;
- $d = \exp(r\lambda - \sigma\sqrt{\lambda})$;

$$p^* = (\exp(rh) - d) / (u - d)$$

Учсть, что

- Пусть $S = 100$, $T = 1$, $n = 10000$, $\sigma = 0.3$ и $r = 0.08$. Попробуйте построить траекторию курса акций. Функция `rand()` генерирует случайное число от 0 до 1. Вы можете использовать функцию построения графика из библиотеки графиков.
- Создайте функцию `createPath(S: Float64, r: Float64, sigma: Float64, T: Float64, n: Int64)`, которая создаст траекторию цены акции с учетом начальных параметров. Используйте `createPath`, чтобы создать 10 разных траекторий и построить их все на одном графике.
- Распараллельте генерацию траекторий. Можете использовать `Threads.@threads`, `spawn` и `@parallel`.
- Пусть $S = 100$, $T = 1$, $n = 10000$, $\sigma = 0.3$ и $r = 0.08$. Попробуйте построить траекторию курса акций. Функция `rand()` генерирует случайное число от 0 до 1.

Рис. 29: Описание задания

Выполнение лабораторной работы

```
[70]: function createPath(S::Float64, r::Float64, sigma::Float64, T::Float64, n::Int64)
      h = T / n
      u = exp(r * h + sigma * sqrt(h))
      d = exp(r * h - sigma * sqrt(h))
      p_star = (exp(r * h) - d) / (u - d)

      # Initialize path
      path = Vector{Float64}(undef, n+1)
      path[1] = S

      for i in 2:(n+1)
          if rand() < p_star
              path[i] = path[i-1] * u
          else
              path[i] = path[i-1] * d
          end
      end

      return path
end

# Parameters
S = 100.0
T = 1.0
n = 1000
sigma = 0.3
r = 0.08;
```

Рис. 30: Модель ценообразования биномиальных опционов

Выполнение лабораторной работы

```
[79]: # Генерируем одну траекторию  
path_single = createPath(S, r, sigma, T, n)
```

```
# Строим график  
x_vals = range(0, T, lengthmm+1)  
plot(x_vals, path_single,  
      label="Одна траектория",  
      title="Траектория цены акции (Биномиальная модель)",  
      xlabel="Время (годы)",  
      ylabel="Цена акции",  
      linewidth=1.5,  
      color='blue')  
display(plot!())
```

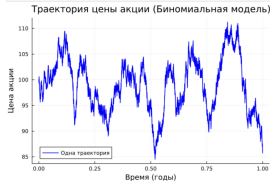


Рис. 31: График траекторий цены акции

Выполнение лабораторной работы

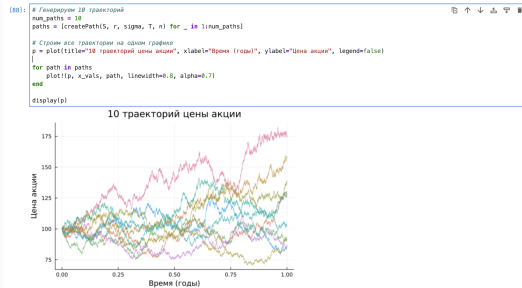


Рис. 32: График для различных траекторий цены

Выполнение лабораторной работы

```
[89]: paths_parallel = Vector(Vector(Float64))(undef, num_paths)

Threads.@threads for i in 1:num_paths
    paths_parallel[i] = createPath(S, r, sigma, T, n)
end

# Строим график
p_parallel = plot(title="10 траекторий (параллельно)", xlabel="Время (годы)", ylabel="Цена акции", legend=false)

for path in paths_parallel
    plot!(p_parallel, x_vals, path, linewidth=0.8, alpha=0.7)
end

display(p_parallel)
```

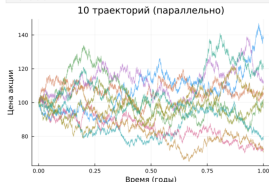


Рис. 33: Параллельное вычисление траекторий

Выводы

В результате выполнения данной лабораторной работы я освоила специализированные пакеты Julia для обработки данных.